

# **ConceptNet Analysis: Knowledge Graph vs. Relational DBMS Comparison**

Data Management Project 2026

Project Group: Valerio Lorito (1885657) & Costanza Maiolino (2064531)

## BACKGROUND

# What is ConceptNet?

**ConceptNet** is a large-scale, multilingual knowledge graph that connects words and phrases through labeled, weighted semantic relationships.

It integrates data from crowdsourced resources (e.g. Open Mind Common Sense), expert-curated databases (e.g. WordNet), and multilingual knowledge bases (e.g. Wiktionary, DBPedia).

Example relationships:

dog —IsA→ animal

tree —PartOf→ forest

knife —UsedFor→ cutting

5.7

Latest Version

36+

Languages

~8M

Nodes

~34M

Edges

## MAIN OBJECTIVE

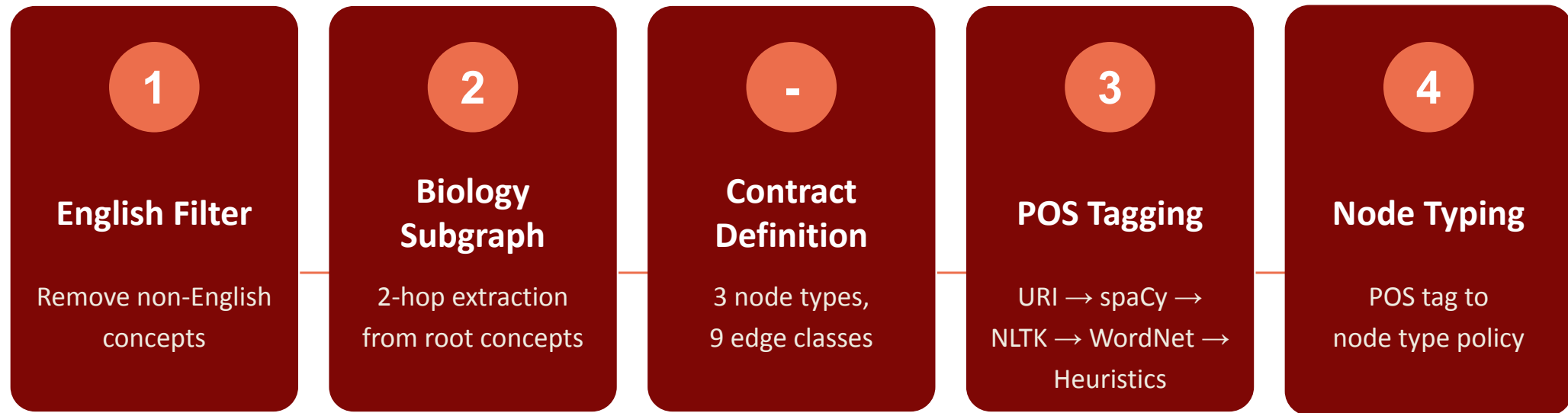
COMPARE TWO DIFFERENT ARCHITECTURES FOR REPRESENTING AND QUERYING CONCEPTNET

SQL-BASED ARCHITECTURE — **MYSQL**

vs.

NOSQL-BASED ARCHITECTURE — **MEMGRAPH**

# Filtering Pipeline — Overview



## FINAL DATASET

481,706 typed edges -> filtered as **contract violations** -> 454,139 loaded  
291,746 typed nodes

*Each step is implemented in Python scripts operating on the full ConceptNet 5.7 dump.*

# Schema Modeling — Node Types

Every concept in the subgraph is assigned to exactly one node type based on its Part-of-Speech tag.

1

## EntityNode

POS TAG 'n' (NOUN)

Concrete or abstract things, substances, organisms, locations, and structures.

e.g. *cell, protein, organism, molecule, forest, enzyme, habitat, tissue, species*

2

## ActionEventNode

POS TAG 'v' (VERB)

Biological processes, actions, events, and state changes.

e.g. *evolve, photosynthesize, digest, replicate, mutate, metabolize, divide, absorb*

3

## PropertyNode

POS TAGS : 'a' (ADJECTIVE)  
'r' (ADVERB)  
's' (SATELLITE ADJECTIVE)

Observable qualities, measurable attributes, and relational descriptors.

e.g. *toxic, aquatic, organic, rapidly, hereditary, microscopic, cellular, parasitic*

*This POS-based classification provides a clean, deterministic partition — every concept maps to exactly one node type with no ambiguity.*

# Schema Modeling — Edge Classes

Each edge is classified into one or more of classes, based on the types of its source and target nodes.

*(Edge classes were determined by analyzing which ConceptNet relations semantically apply between each pair of node types).*

**Entity → Entity**

**12**

IsA, PartOf, MadeOf, DefinedAs, InstanceOf, SymbolOf, FormOf, LocatedNear, AtLocation, EtymologicallyRelatedTo, EtymologicallyDerivedFrom, DerivedFrom

**Entity → Property**

**3**

HasProperty, NotHasProperty, DefinedAs

**Entity → Action**

**7**

CapableOf, UsedFor, ReceivesAction, NotCapableOf, Desires, NotDesires, CreatedBy

**Action → Action**

**7**

HasSubEvent, HasFirstSubevent, HasLastSubevent, HasPrerequisite, Entails, MannerOf, MotivatedByGoal

**Action → Property**

**2**

HasProperty, NotHasProperty

**ALL (any ↔ any)**

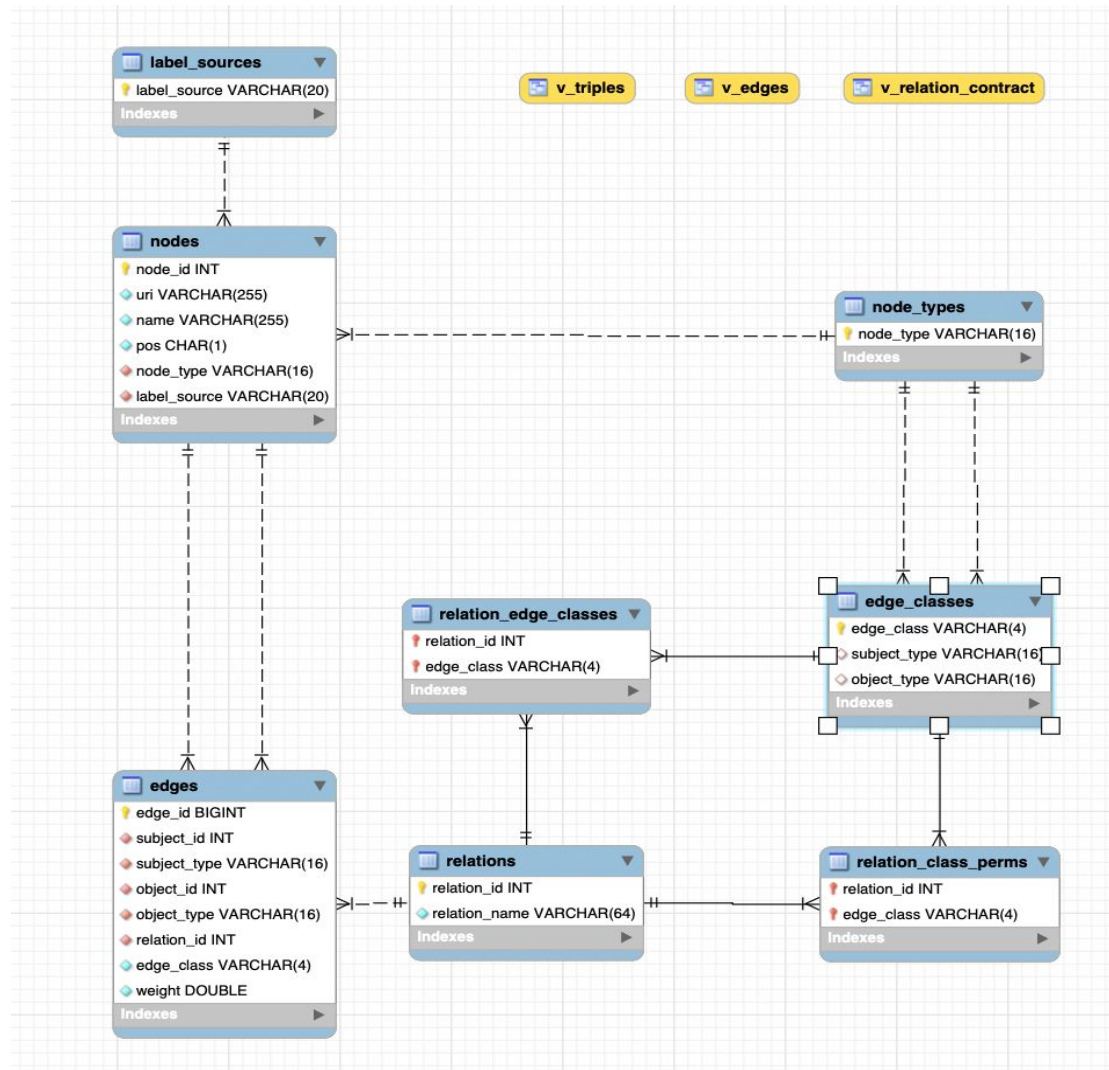
**9**

DBPedia, Causes, CausesDesire, DistinctFrom, RelatedTo, Synonym, Antonym, SimilarTo, HasContext

*The edge-class taxonomy also defines Property→Property, Property→Entity, Property→Action and Action→Entity as valid classes. However, these classes are empty in the final subgraph — these classes carry zero edges in this slice. They remain in the schema for completeness but carry zero edges.*

# Relational Architecture (SQL)

Entity-Relationship modeling for ConceptNet 5.7 structured data.



## Normalized Tables

Strict separation of nodes, edges, and classification taxonomy to minimize redundancy.

## Foreign Key Constraints

Referential integrity enforced natively across all relational mapping tables.

## JOIN-Dependent Traversals

Multi-hop paths require computationally expensive nested JOIN operations.

# Native Graph Architecture (Memgraph)

Property graph model optimized for semantic relationship traversals. Visually, a giant mess...

## First-Class Relationships

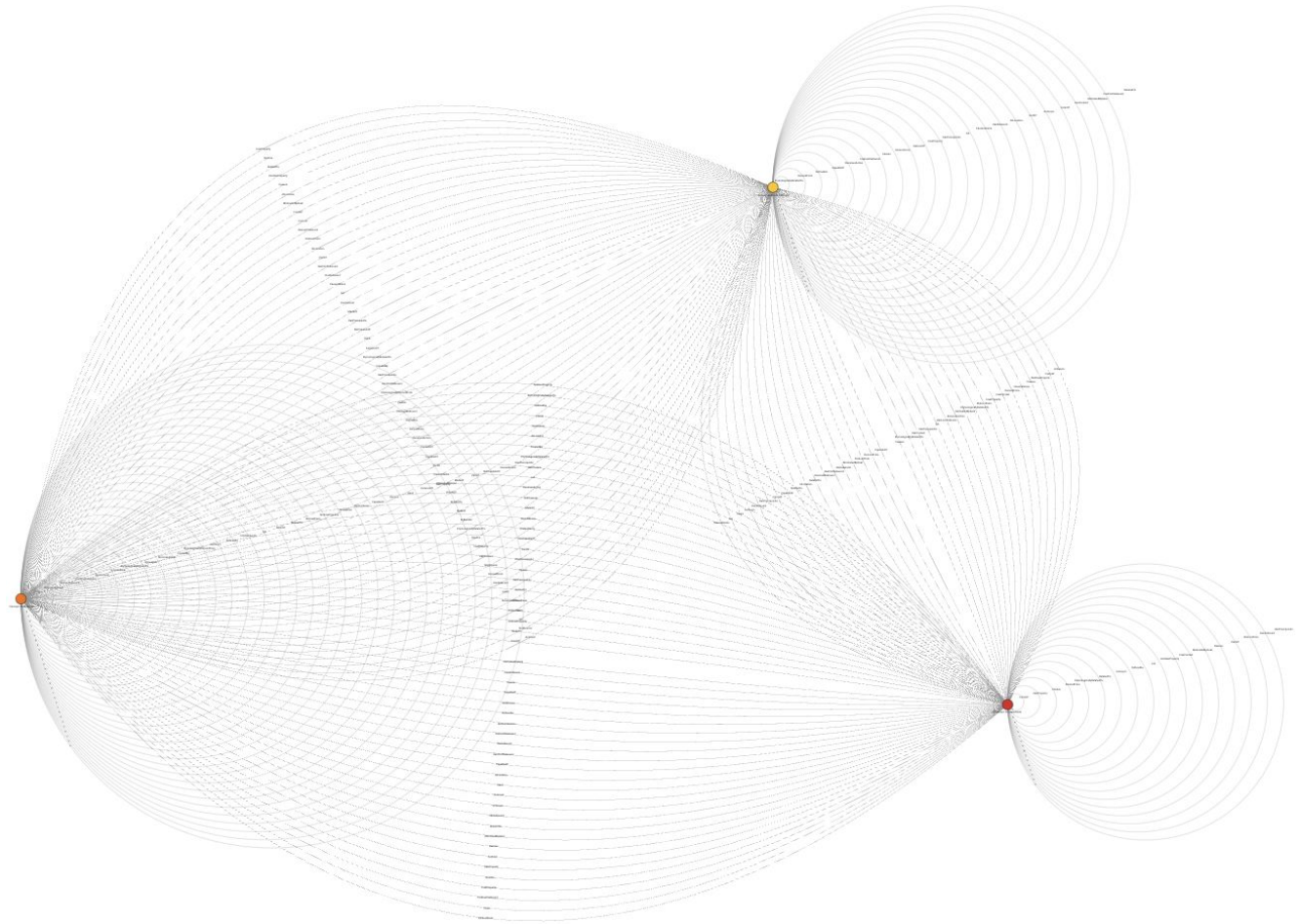
Edges are stored directly as native entities, not computed at query time.

## Index-Free Adjacency

Nodes hold direct memory pointers to connected neighbors for  $O(1)$  hops.

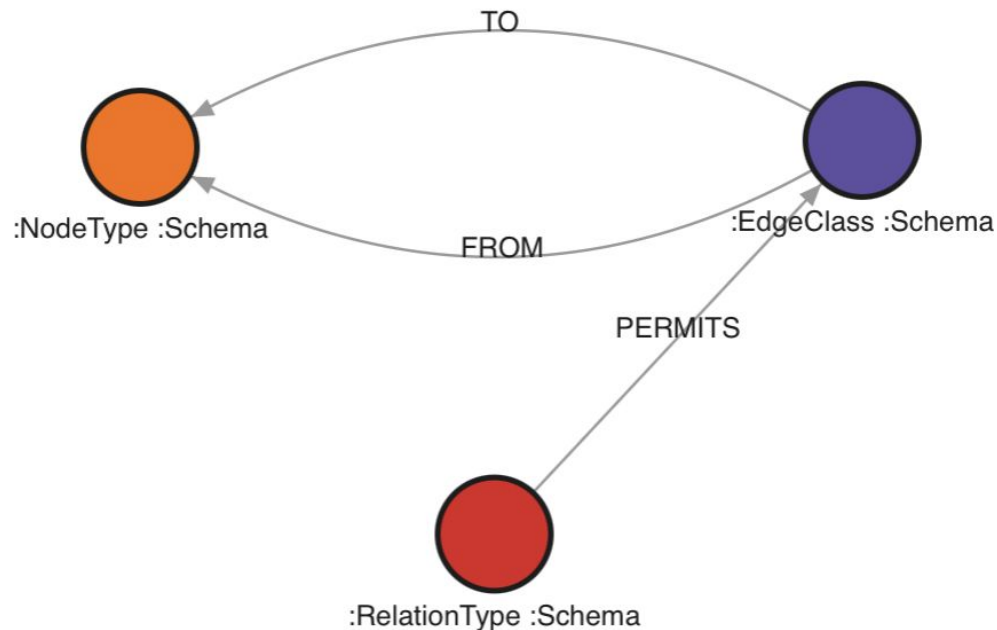
## Schema Flexibility

Properties attached dynamically to nodes and edges without rigid table definitions.



# Memgraph Permission Meta-Graph

Since Memgraph has no built-in schema enforcement, the edge-class contract is encoded as a separate graph structure — queryable at validation time.



**Example:**

*IsA* —PERMITS→ *E2E*

| —FROM→ *EntityNode*

|

| —TO→ *EntityNode*

## The Need of a Permission Schema

Memgraph accepts any edge between any nodes — there are no CHECK constraints, no foreign keys, no endpoint-type rules on relations. An invalid edge (e.g. HasProperty between two EntityNodes) is silently inserted.

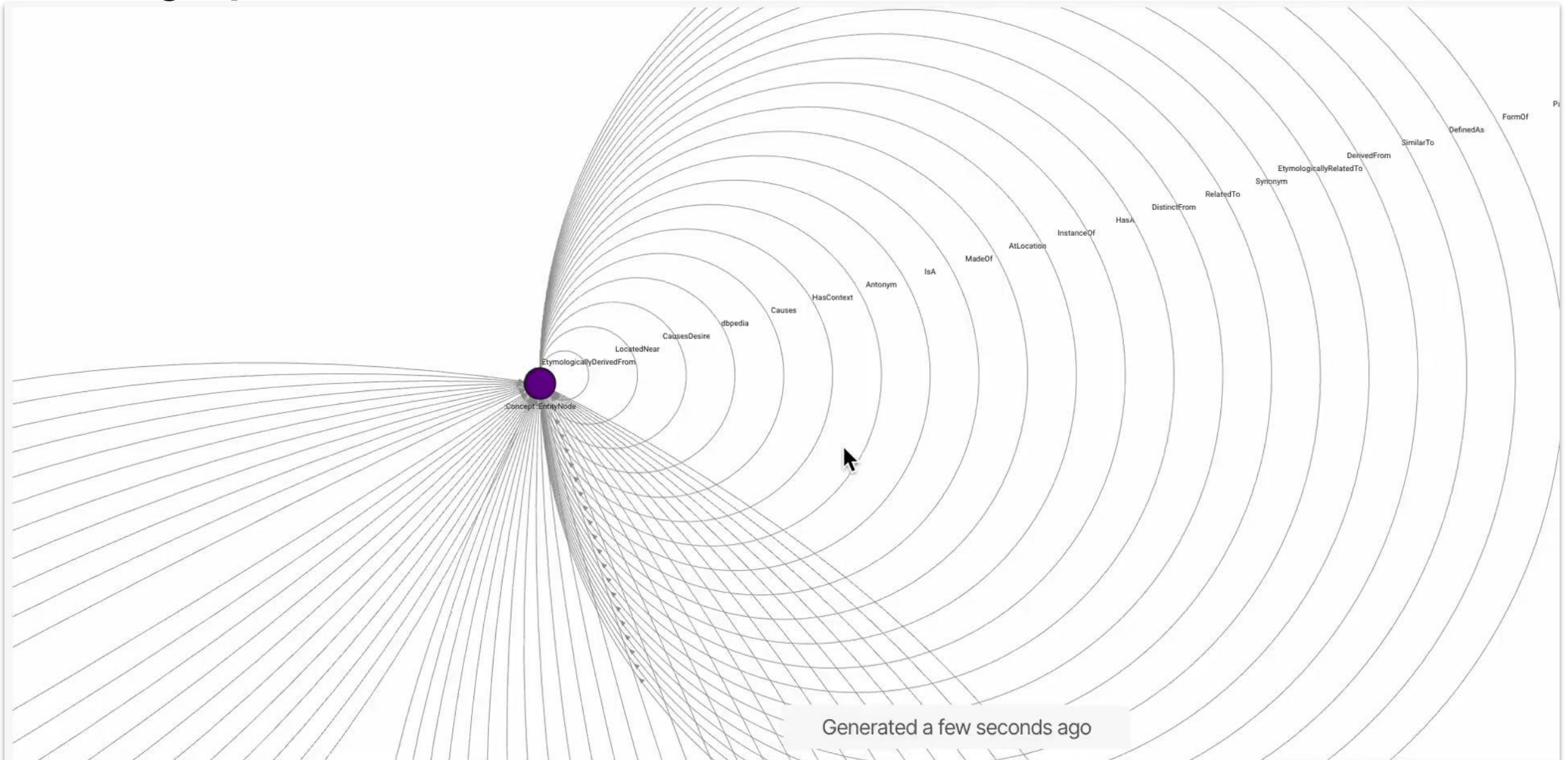
## Violation Detection at Query-Time

This meta-graph acts as a declarative permission layer that can be queried to detect violations.

## Differences from MySQL

In MySQL, this contract is intrinsic to the relational schema: CHECK constraints validate edge classes at insertion, composite foreign keys enforce endpoint types, and the `v_edges` view flags violations automatically.

# Memgraph Architecture:



# Comparative Query Analysis

We designed four query families to evaluate different categories of inference that can be performed on a knowledge graph, testing both architectures on information retrieval capabilities and schema integrity enforcement.

Every result was verified: result-set hashes were compared across engines to ensure functional equivalence, and isomorphic loads were asserted per run to guarantee both architectures operated on identical data. Cross-verification surfaced five classes of silent error (query-text desyncs, grouping ambiguity, pattern-semantics differences, planner access paths, contract drift) — each fixed and viewable..

## Analysis Objectives

- Measure the overhead of SQL JOINS vs graph pattern matching on equivalent queries.
- Compare B-Tree indexed lookups (MySQL) against in-memory adjacency pointers (Memgraph) for local retrieval operations.
- Evaluate how execution time scales with traversal depth — identifying the crossover point where graph-native traversal outperforms relational joins.
- Test queries that expose expressiveness gaps — operations native to Cypher with no practical SQL equivalent.
- Assess full-graph aggregation throughput, where both engines must scan the entire edge set.
- Verify schema integrity enforcement: preventive (MySQL CHECK + FK at write time) vs detective (Memgraph meta-graph validation at query time).
- Compare referential integrity handling on write operations — insertion of invalid data and cascading deletions.

# Family A: Local Retrieval

Evaluates basic read operations, indexing efficiency, and single-hop adjacency. Designed to test the baseline overhead of SQL JOINS against direct memory pointers.

**Node Searching**  
Exact URI matches.  
Alphabetical range scans.

**1-Hop Expansion**  
Targeted relation queries.  
Bidirectional traversal tracking.

**Pattern Intersections**  
Shared neighbor identification.  
Relational set difference.

- A1 Single Node Search** — Find a concept by its URI — primary key index lookup.
- A6 Range Selection** — All concepts whose name falls within a lexicographic range (e.g. 'cell' to 'gene').
- A2 Full Relation Table** — All outgoing relations of a concept, ranked by weight.
- A3 Relation 1-hop** — Follow a specific relation to a target node type (e.g. cell→CapableOf→Action).
- A4 Directed vs. Undirected** — Compare incoming-edge lookup with bidirectional traversal on symmetric relations.
- A5 Relation Pair Intersection** — Concepts related to both A and B via the same relation type.
- A7 Neighborhood Difference** — Concepts related to A but not to B — tests anti-join / NOT EXISTS.

# Local Retrieval — Results

SQL wins on most local queries thanks to B-Tree indexing and optimized JOINS.  
Memgraph advantage appears on undirected traversals (A4b) and set-difference patterns (A7) where Cypher's syntax maps directly to the operation.

Differences under ~1–2 ms reported as ties; medians over N runs; all result sets hash-verified.

| Query | SQL (ms) | Cypher (ms) | Winner | Analysis                                                                                                                      |
|-------|----------|-------------|--------|-------------------------------------------------------------------------------------------------------------------------------|
| A1    | 0.25     | 32.34       | SQL    | URI lookup: SQL 94x faster - Memgraph's property index wasn't used for this lookup.                                           |
| A2    | 0.51     | 1.64        | SQL    | Relation table: SQL 2.8x faster — JOIN chain well-optimized by MySQL planner.                                                 |
| A3    | 0.39     | 0.95        | Tie    | 1-hop typed: near tie — both engines handle single-hop efficiently.                                                           |
| A4a   | 0.47     | 0.93        | Tie    | Directed (reverse): SQL slightly faster on incoming-edge indexed lookup.                                                      |
| A4b   | 5.54     | 1.08        | Graph  | Undirected: Memgraph 3.4x faster — bidirectional pattern matching is native.                                                  |
| A5    | 3.70     | 1.19        | Graph  | Pair intersection: Memgraph 3.1x faster - the pattern expresses the join topology directly; SQL pays the two-branch self-join |
| A6    | 133.80   | 728.32      | SQL    | Range scan (60k rows): SQL faster — B-Tree range scans are SQL's sweet spot.                                                  |
| A7    | 1.23     | 0.46        | Graph  | Set difference: Memgraph wins — NOT EXISTS is more natural in Cypher.                                                         |

## Family B: Traversal Paths

Multi-hop explorations, shortest paths, and recursive traversals. Tests the core trade-off: SQL recursive CTEs with manual cycle detection versus Cypher's native variable-length patterns.

### Multi-Hop Exploration

Weight-filtered neighborhood.  
Hierarchical IsA chains.

### Path Finding

BFS shortest path.  
Weight-product path scoring.

### Structural Patterns

Cycle / triangle detection.  
Neighbor degree profiling.

- B1 K-Hop Neighborhood** — All concepts reachable in K hops, filtered by minimum edge weight.
- B2 Transitive Closure** — Follow IsA chains up to depth 5 — find all superclasses of a concept.
- B3 Shortest Chain** — Shortest undirected path between two concepts via BFS (any relation).
- B4 Weighted Path Scoring** — Rank paths by product of edge weights. No SQL equivalent exists.
- B5 Anchored Triangle** — 3-hop cycles starting and ending at the same concept (structural motif).
- B6 Degree Profile** — For each neighbor, count its outgoing degree — local centrality analysis.

# Traversal Paths — Results

*The K-hop crossover at k=4 is the headline result: SQL wins at shallow depth, but Memgraph's O(1) pointer traversal scales better. For path-finding (B3), Memgraph is 16× faster and B4 has no SQL equivalent — the strongest argument for a graph engine on this workload.*

Median execution time (ms). B4 excluded (no SQL counterpart — Memgraph executes in 3.55 ms since path queries are natively efficient).

| Query | SQL (ms) | Cypher (ms) | Winner | Analysis                                                                                                 |
|-------|----------|-------------|--------|----------------------------------------------------------------------------------------------------------|
| B1k1  | 0.37     | 0.43        | Tie    | 1-hop weighted: near tie - minimal traversal, both fast.                                                 |
| B1k2  | 0.38     | 0.51        | Tie    | 2-hop: SQL still ahead - JOINS competitive at shallow depth.                                             |
| B1k3  | 0.54     | 0.93        | SQL    | 3-hop: gap narrowing - SQL's JOIN cost growing with depth.                                               |
| B1k4  | 3.78     | 2.32        | Graph  | 4-hop: crossover - Memgraph overtakes as index-free adjacency pays off.                                  |
| B2    | 0.33     | 0.40        | Tie    | Transitive closure (IsA, depth 5): near tie - recursion frontier stays small, so recursive CTE is cheap. |
| B3    | 1638.09  | 89.96       | Graph  | Shortest path BFS: Memgraph 16.6× faster - native BFS vs. recursive CTE.                                 |
| B5    | 18.81    | 19.64       | Tie    | Anchored triangle (3-hop cycle) — near tie, both handle triple-JOIN well.                                |
| B6    | 15.05    | 6.01        | Graph  | Degree profile: Cypher slightly faster — traversal feeds aggregation without JOINS.                      |

*Cypher advantage: path queries (B3, B4) are native 1-liners — SQL requires recursive CTEs, JSON path tracking, and UNION ALL.*

## Family C: Global Retrieval

---

Full-graph aggregations scanning the entire dataset. Measures raw throughput on large GROUP BY operations — where both engines need to touch every edge and the performance gap narrows.

### Degree Analytics

Full edge-set scan.

Top-K ranking by out-degree.

### Schema Cross-Tabulation

Per-relation, per-class aggregation.

Edge count + average weight matrix.

**C1** **Concept Hubs Ranking** — Top-10 concepts by outgoing edge count — full GROUP BY on all edges.

**C2** **Relation-Class Matrix** — Group all edges by (relation, edge\_class) — cross-tab of the entire schema.

## Family D: Schema & Integrity

---

Constraint enforcement, violation detection, and data mutation. Evaluates how each architecture handles schema validation at write time and referential integrity during deletions.

### Constraint Validation

A priori type-contract check.

Preventive vs. detective approach.

### Data Mutation

Cascading deletion.

Referential integrity handling.

**D1** **Violation Check** — Count edges violating the type contract. MySQL uses v\_edges view; Memgraph queries :Schema meta-graph.

**D2** **Invalid Insert** — Insert a contract-violating edge. MySQL rejects (CHECK + FK); Memgraph accepts silently.

**D3** **Concept Delete** — Remove a concept and all incident edges. MySQL: ordered DELETE (FK order); Memgraph: DETACH DELETE.

*Key insight: MySQL enforces schema at write time (preventive) — Memgraph relies on query-time validation (detective).*

# Global Retrieval & Schema Integrity

Median execution time (ms). All results verified OK.

| Q  | SQL    | Cypher  | Win   | Key Finding                                                                                                                                                                                              |
|----|--------|---------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| C1 | 894.57 | 555.87  | Graph | Hub ranking — Memgraph 1.6× faster. SQL pays join-backs to nodes / relations.                                                                                                                            |
| C2 | 576.22 | 163.68  | Graph | Relation-class matrix — Memgraph 3.5× faster. Same reason as C1 analysis.                                                                                                                                |
| D1 | 67.44  | 4556.20 | SQL   | Violation check — MySQL checks the contract via a view over reference table, while Memgraph runs a per-edge anti-join.                                                                                   |
| D2 | 75.20  | 4664.76 | SQL   | Invalid Insert -MySQL rejects 2 of 3 invalid inserts at write time (CHECK 3819, FK 1452); the third (truthful class, unpermitted combo) loads flagged 'permitted=0'. Memgraph accepts all three silently |
| D3 | 2.45   | 30.39   | SQL   | Concept delete — MySQL fails at deleting the parent first (FK 1451; children must go first). Memgraph uses one atomic 'DETACH DELETE', no order needed.                                                  |

## Qualitative Observations

**Family C — Graph wins both**

In-memory iteration over all 454k edges is faster than disk-based InnoDB page reads. The gap widens on C2 (3.5×) where Memgraph's label-based CASE avoids the JOIN to the relations table.

**Family D — SQL wins all three**

MySQL enforces constraints at write time (preventive): CHECK + FK reject invalid data instantly. Memgraph accepts silently and validates retroactively via the :Schema meta-graph — ~68× slower.

Memgraph dominates global aggregations (C). MySQL dominates schema enforcement (D) — a fundamental architectural difference.

# MySQL vs Memgraph — Final Verdict

## MySQL

### Excels when the workload is structured

- **Point & range access:** B+ tree equality and range scans dominate local retrieval — shallow joins are cheap for the planner.
- **Integrity at write time:** CHECK + composite FKs reject invalid data on insert (D2: 3819/1452) and prevent orphaning on delete (D3: 1451) — no query needed.
- **Contract as catalog:** the type contract is a declared schema; checking it is a view over reference tables — 68× faster than query-time validation (D1).
- **Durable & compact:** 186 MB on disk with persistence by default — vs. 1.54 GiB resident in RAM.

## Memgraph

### Excels when the workload is navigational

- **Deep traversal:** index-free adjacency wins as hops compound — crossover at  $k = 4$  (B1), 16× on shortest path (B3)
- **Path-shaped questions:** variable-length patterns are native; B4 has no SQL formulation— recursive CTEs only approximate it.
- **Undirected & difference patterns:** one pattern where SQL needs OR-predicates or anti-joins (A4b, A7).
- **Full-graph aggregation:** reads labels/properties off traversed records — no join-backs to reassemble tuples (C1, C2: up to 3.5×).

**Bottom line:** Neither architecture dominates: the winner is decided by the shape of the question. Local lookups, declared integrity, and compact durability favor MySQL; multi-hop navigation, path semantics, and whole-graph aggregation favor Memgraph — and only the relational engine can *guarantee* the contract, while only the graph engine treats paths as data.



THANK  
YOU!!!